

A FAST FOURIER TRANSFORM BASED DIRECT SOLVER FOR THE HELMHOLTZ PROBLEM

JARI TOIVANEN AND MONIKA WOLFMAYR

ABSTRACT. This paper is devoted to the efficient numerical solution of the Helmholtz equation in a two- or three-dimensional orthogonal domain with an absorbing boundary condition (ABC). The Helmholtz problem is discretized by standard bilinear and trilinear finite elements on an orthogonal mesh yielding a separable system of linear equations. The main key to high performance is to employ the Fast Fourier transform (FFT) within a fast direct solver to solve the large separable systems. Numerical results for both two- and three-dimensional problems are presented confirming the efficiency of the method discussed.

1. INTRODUCTION

This work presents an efficient fast direct solver employing FFT for the Helmholtz equation

$$(1) \quad -\Delta u - \omega^2 u = f$$

in a rectangular domain with absorbing boundary conditions (ABCs). The method can be applied for problems with the constant zeroth order term coefficient ω being positive, negative, or zero. Here, we focus on large indefinite Helmholtz problems as they are a common result from acoustic scattering problems. The zeroth order term coefficient ω denotes the wave number, which is assumed to be a constant. This condition is valid for homogeneous media. For example, the iterative domain decomposition solution techniques for acoustic scattering in layered media considered in [8, 13] leads a sequence of such problems. In this work, we derive the fast solver for the case of having a first-order ABC imposed. However, the same method can be used for the second-order ABCs employed in [11]. These ABCs are the time-harmonic counterpart of the ABCs for the wave equation considered in [1]. Moreover, the solver can be used also for problems with Robin boundary conditions, which essentially broadens its applicability. Other separable Robin boundary conditions can be treated the same way as the ABCs, which are just one possible type of boundary conditions for the Helmholtz equation being solved by the method discussed in this work. Solving the Helmholtz equation is in general difficult or impossible to solve efficiently with most numerical methods. Difficulties related to the numerical solution of time-harmonic Helmholtz equations are discussed for instance in [20].

The numerical method proposed in this work is a fast direct solver, which is applicable for problems posed in rectangular domains and having suitable tensor product form matrices. This kind of diagonalization technique has already been proposed in [15]. However, we use FFT and sparsity in order to implement it efficiently. The method is applicable for any discretization leading to separable 9-point stencil for two-dimensional problems and 27-point stencil for three-dimensional problems. For example, bilinear or trilinear finite element discretizations employed in this paper lead to matrices with such suitable tensor product form. Also the fourth-order accurate modified versions of these elements [7] are applicable with the solver. When using fourth-order accurate bilinear or trilinear finite elements, the reduced pollution error is obtained. To improve the efficiency this method employs FFT instead of cyclic reduction. Several other fast direct solvers for elliptic problems in rectangular domains are discussed, e.g., in [18].

The fast direct solvers can be used as efficient preconditioners in the iterative solution of problems in more general domains, see [3, 12]. Other methods, which are based on an equivalent

This research was supported by the Academy of Finland, grant 295897.

formulation of the original problem to enable preconditioning with fast direct methods, are referred to as fictitious domain, domain imbedding, or capacitance matrix methods, and they have been successfully applied also to acoustic scattering problems for instance in [6, 9, 10, 12] as well as to scattering problems with multiple right-hand sides, where a specific method for such problems is considered, for example, in [19].

A different fast direct solver for the Helmholtz equation (1) has already been presented in [11], where cyclic reduction techniques are used in the solution procedure. In [11], the application of a specific cyclic reduction-type fast direct method is presented, called the partial solution variant of the cyclic reduction (PSCR) method, to the solution of the Helmholtz equation [16, 17, 21] and its computational efficiency is demonstrated. For three-dimensional problems, a general fast direct solver like the PSCR has excellent parallel scalability, see [17]. However, the use of FFT over cyclic reduction techniques is preferable, since FFT is generally faster. That is partly due to very fast implementations of FFT.

The reason for not applying FFT previously was that the ABCs prevent the diagonalization using FFT. The following three steps describe the basic idea to solve this problem and are discussed in more detail in this work:

- (1) Some of the boundary conditions are changed on the ABC part of the boundary to be of periodic type. This modified problem can be solved now with an FFT solver.
- (2) Since the boundary conditions have been changed to periodic ones, the residual vector is computed due to these incorrect boundary conditions. This has nonzero components only on the boundary parts, where we have changed the boundary conditions. The correction is computed by solving a problem with the original matrix, where the right-hand side vector is the residual. Only the component of the correction which lie on the changed boundaries are computed. For this the so-called partial solution problem a special technique exists, see [2, 11, 14, 17] for example. Due to sparsity of the vectors the solution requires only $\mathcal{O}(N)$ or $\mathcal{O}(N \log N)$ operations in case of two or three dimensions, respectively, where N is the total number of unknowns. After this step, the correct boundary values are known.
- (3) A similar problem to the one in the first step is solved, but now the right-hand side vector is adjusted so that the solution has correct values on the boundaries. From this it follows that the solution is also the solution of the original problem.

A similar method to the above one was already proposed in [5], but with the major difference that the problem in the second step was solved iteratively. The new solver employing the FFT method is efficient in terms of computational time and memory usage, especially in the three-dimensional case. For three-dimensional problems, a general fast direct solver like the PSCR method [17] requires $\mathcal{O}(N(\log N)^2)$ operations, whereas the FFT based solver reduces the complexity to $\mathcal{O}(N \log N)$ operations.

The paper is organized as follows: In Section 2, we present the classic and variational formulation of the Helmholtz problem as well as its discretization by bilinear and trilinear finite elements leading to a system of linear equations. The main idea of the solver for this problem and some preliminaries, which appear in the initialization process of the implemented fast solver for the two- and three-dimensional case, are discussed in Section 3. Sections 4 and 5 are devoted to the two- and three-dimensional problems, respectively. Here, some preliminaries as well as the fast solver steps are discussed. Finally, numerical results for both two- and three-dimensional problems are presented in Section 6, and conclusions are drawn in Section 7.

2. PROBLEM FORMULATION AND DISCRETIZATION

Let $\Omega \subset \mathbb{R}^d$, $d = 2, 3$, be a d -dimensional rectangular domain and $\Gamma = \partial\Omega$ its boundary. We consider the Helmholtz equation describing the linear propagation of time-harmonic acoustic waves given by

$$\begin{aligned} (2) \quad & -\Delta u - \omega^2 u = f \quad \text{in } \Omega, \\ (3) \quad & \mathcal{B}u = 0 \quad \text{on } \Gamma, \end{aligned}$$

where ω denotes the wave number. Here, equation (3) is an approximation for the Sommerfeld radiation condition

$$(4) \quad \lim_{r \rightarrow \infty} r^{(d-1)/2} (\partial_r u - i\omega u) = 0$$

appearing for the general model problem in $\mathbb{R}^d$. The approximation is performed by truncating the unbounded domain at a finite distance, which provides the boundary condition at the truncation boundary. In order to reduce spurious reflections caused by this artificial boundary, we use local absorbing boundary conditions. In general, the boundary Γ can be decomposed into parts where different boundary conditions are imposed. Let now $\Gamma = \Gamma_B \cup \Gamma_N$ be decomposed into an absorbing boundary condition part Γ_B and a Neumann boundary condition part Γ_N . The Neumann boundary conditions are given by

$$(5) \quad \mathcal{B}u = \nabla u \cdot \mathbf{n} = 0 \quad \text{on } \Gamma_N,$$

whereas in case of absorbing boundary conditions we have that

$$(6) \quad \mathcal{B}u = \nabla u \cdot \mathbf{n} - i\omega u = 0 \quad \text{on } \Gamma_B,$$

where $\mathbf{n}$ denotes the outward normal to the boundary. This type of absorbing boundary conditions (6) are called first-order absorbing boundary conditions.

Remark 1. *In practical applications, so-called second-order absorbing boundary conditions are also important and were considered in [11] as well. However, the choice of the absorbing boundary conditions does not have any impact on the performance of the proposed solver. Thus, the same method can be used with given second-order absorbing boundary conditions, see [11].*

In order to obtain the discrete version of the Helmholtz problem (2)–(3), let us first state its weak formulation. For that we introduce the Hilbert space $V = H^1(\Omega)$. Let the source term $f \in L_2(\Omega)$ be given. Multiplying (2) with a test function as well as applying integration by parts and the boundary condition (3), yields the weak problem: Find $u \in V$ such that

$$(7) \quad a(u, v) = \int_{\Omega} f v \, d\mathbf{x} \quad \forall v \in V,$$

where

$$(8) \quad a(u, v) = \int_{\Omega} (\nabla u \cdot \nabla v - \omega^2 uv) \, d\mathbf{x} - i\omega \int_{\partial\Omega} uv \, ds.$$

Let us denote the mesh points by $x_{j,l}$, $l = 1, \dots, n_j$ for every x_j -direction with $j \in \{1, \dots, d\}$ and the corresponding mesh size by $h_j = 1/(n_j - 1)$. Thus, the mesh is equidistant in each direction x_j . Moreover, we denote the full mesh size by N , which is given by $N = n_1 \times \dots \times n_d$. Discretizing problem (7) by bilinear or trilinear finite elements on an orthogonal mesh leads to a system of linear equations given by

$$(9) \quad \mathbf{A} \mathbf{u} = \mathbf{f},$$

where the matrix $\mathbf{A}$ has a separable tensor product form and $\mathbf{f} = (f_1, \dots, f_N)$ denotes the discrete right-hand side. For the two-dimensional case, the matrix $\mathbf{A}$ is given by

$$(10) \quad \mathbf{A} = (\mathbf{K}_1 - \omega^2 \mathbf{M}_1) \otimes \mathbf{M}_2 + \mathbf{M}_1 \otimes \mathbf{K}_2,$$

whereas in three dimensions it is given by

$$(11) \quad \mathbf{A} = (\mathbf{K}_1 - \omega^2 \mathbf{M}_1) \otimes \mathbf{M}_2 \otimes \mathbf{M}_3 + \mathbf{M}_1 \otimes (\mathbf{K}_2 \otimes \mathbf{M}_3 + \mathbf{M}_2 \otimes \mathbf{K}_3).$$

Here, the $n_j \times n_j$ -matrices $\mathbf{K}_j$ and $\mathbf{M}_j$ are one-dimensional stiffness and mass matrices, respectively, in the x_j -direction with possible modifications on the boundaries due to the absorbing boundary conditions. They are computed by one-dimensional numerical quadrature on the unit

interval and are given as follows

$$(12) \quad \mathbf{K}_j = \frac{1}{h_j} \begin{pmatrix} k_{1,1} & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & k_{n_j, n_j} \end{pmatrix}$$

and

$$(13) \quad \mathbf{M}_j = \frac{h_j}{6} \begin{pmatrix} 2 & 1 & & & \\ 1 & 4 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & 4 & 1 \\ & & & 1 & 2 \end{pmatrix}$$

where the first and last entries are including the corresponding boundary conditions. Absorbing boundary conditions (6) yield the entries $k_{1,1} = k_{n_j, n_j} = 1 - i\omega h_j$, whereas Neumann boundary conditions lead to $k_{1,1} = k_{n_j, n_j} = 1$. The matrix $\mathbf{M}_j$ is the same for both Neumann and (first-order) absorbing boundary conditions.

Remark 2. *Without loss of generality let us assume that the absorbing boundary conditions are posed in direction of x_1 for both (opposite) sides.*

In the next two sections, an efficient fast solver for solving the large linear systems $\mathbf{A}\mathbf{u} = \mathbf{f}$ is proposed. We split the discussion into two sections corresponding to the two-dimensional and three-dimensional problems with the matrix $\mathbf{A}$ given by (10) and (11), respectively.

3. MAIN IDEA AND PRELIMINARIES

3.1. Basic steps. The idea for solving the problem $\mathbf{A}\mathbf{u} = \mathbf{f}$ is to consider an auxiliary problem

$$(14) \quad \mathbf{B}\mathbf{v} = \mathbf{f},$$

where the system matrix $\mathbf{B}$ is derived by changing some absorbing boundary conditions to periodic ones. The key is that we can solve the modified (periodic) problem $\mathbf{B}\mathbf{v} = \mathbf{f}$ now by using the FFT method, which is not possible for the original problem $\mathbf{A}\mathbf{u} = \mathbf{f}$.

Before going into more details later, let us discuss the main steps of the solver.

Step 1: Solve problem (14) $\mathbf{B}\mathbf{v} = \mathbf{f}$.

Step 2: Let $\mathbf{w} = \mathbf{u} - \mathbf{v}$. Solve

$$(15) \quad \mathbf{A}\mathbf{w} = \mathbf{f} - \mathbf{A}\mathbf{v} = \mathbf{B}\mathbf{v} - \mathbf{A}\mathbf{v} = (\mathbf{B} - \mathbf{A})\mathbf{v}.$$

Step 3: Solve

$$(16) \quad \mathbf{B}\mathbf{u} = \mathbf{f} + (\mathbf{B} - \mathbf{A})(\mathbf{v} + \mathbf{w}).$$

Note that after Step 2, we would have already had the identity $\mathbf{u} = \mathbf{v} + \mathbf{w}$, but we do not use this identity directly to compute $\mathbf{u}$, which will be explained in Subsections 4.2 and 5.2.

3.2. Preliminaries. We have assumed that the absorbing boundary conditions are given in x_1 -direction on both opposite boundaries, see Remark 2. Since we have to change the boundary conditions on the two opposite boundaries to be of periodic type for the auxiliary problem (14), we consider the one-dimensional stiffness and mass matrices $\mathbf{K}_1$ and $\mathbf{M}_1$ and change the entries corresponding to the boundary parts into periodic conditions for the auxiliary problem (14). We denote these matrices by $\mathbf{K}_1^B$ and $\mathbf{M}_1^B$. The $n_j \times n_j$ circulant matrices $\mathbf{K}_j^B$ and $\mathbf{M}_j^B$ are given

as follows

$$(17) \quad \mathbf{K}_j^B = \frac{1}{h_j} \begin{pmatrix} 2 & -1 & & -1 \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ -1 & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix}$$

and

$$(18) \quad \mathbf{M}_j^B = \frac{h_j}{6} \begin{pmatrix} 4 & 1 & & 1 \\ 1 & 4 & 1 & \\ & \ddots & \ddots & \ddots \\ & & 1 & 4 & 1 \\ 1 & & & 1 & 4 \end{pmatrix}$$

which means that we have changed the boundary conditions on two opposite boundaries to be of periodic type. Hence,

$$(19) \quad \begin{aligned} \mathbf{K}_{j,(1,2)}^B &= \mathbf{K}_{j,(1,n_j)}^B = \mathbf{K}_{j,(2,1)}^B = \mathbf{K}_{j,(n_j,1)}^B, \\ \mathbf{M}_{j,(1,2)}^B &= \mathbf{M}_{j,(1,n_j)}^B = \mathbf{M}_{j,(2,1)}^B = \mathbf{M}_{j,(n_j,1)}^B. \end{aligned}$$

Let us consider now the original problem (9) and the auxiliary problem (14) in more detail. After a suitable permutation $\mathbf{A}$ and $\mathbf{B}$ have the block forms

$$(20) \quad \mathbf{A} = \begin{pmatrix} \mathbf{A}_{bb} & \mathbf{A}_{br} \\ \mathbf{A}_{rb} & \mathbf{A}_{rr} \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} \mathbf{B}_{bb} & \mathbf{A}_{br} \\ \mathbf{A}_{rb} & \mathbf{A}_{rr} \end{pmatrix},$$

the subscripts b and r correspond to the nodes on the Γ_B boundary and to the rest of the nodes, respectively. We denote by $|b|$ and $|r|$ the corresponding sizes of the set of nodes on the Γ_B boundary and of the rest of the nodes such that $N = |b| + |r|$. Note that the matrix $\mathbf{B} - \mathbf{A}$ has the structure

$$(21) \quad \mathbf{B} - \mathbf{A} = \begin{pmatrix} \mathbf{B}_{bb} - \mathbf{A}_{bb} & 0 \\ 0 & 0 \end{pmatrix},$$

and only the matrix

$$(22) \quad \mathbf{C}_{bb} = \mathbf{B}_{bb} - \mathbf{A}_{bb}$$

has to be saved for the application of the fast solver. Steps of the fast solver for $\mathbf{A}\mathbf{u} = \mathbf{f}$ include also the application of the so-called partial solution method, which is a special implementation of the method of separation of variables. This method involves the solution of the generalized eigenvalue problems

$$(23) \quad \mathbf{K}_1 \mathbf{V}_1 = \mathbf{M}_1 \mathbf{V}_1 \mathbf{\Lambda}_1^A$$

and

$$(24) \quad \mathbf{K}_1^B \mathbf{W}_1 = \mathbf{M}_1^B \mathbf{W}_1 \mathbf{\Lambda}_1^B,$$

where the matrices $\mathbf{\Lambda}_1^A$ and $\mathbf{\Lambda}_1^B$ contain the eigenvalues as diagonal entries and the matrices $\mathbf{V}_1$ and $\mathbf{W}_1$ contain the corresponding eigenvectors as their columns. Let us define the more general problem

$$(25) \quad \mathbf{K}_j^B \mathbf{W}_j = \mathbf{M}_j^B \mathbf{W}_j \mathbf{\Lambda}_j^B$$

with circulant matrices $\mathbf{K}_j^B$ and $\mathbf{M}_j^B$ for the problem size n_j corresponding to any x_j -direction. The $n_j \times n_j$ eigenvector matrix $\mathbf{W}_j$ for the generalized eigenvalue problem (25) is given by

$$(26) \quad \mathbf{W}_j = \begin{pmatrix} 1 & 1 & 1 & \dots & 1 & 1 \\ 1 & e^{-\frac{2\pi i}{n_j}} & e^{-2\frac{2\pi i}{n_j}} & \dots & e^{-(n_j-2)\frac{2\pi i}{n_j}} & e^{-(n_j-1)\frac{2\pi i}{n_j}} \\ 1 & e^{-2\frac{2\pi i}{n_j}} & e^{-4\frac{2\pi i}{n_j}} & \dots & e^{-2(n_j-2)\frac{2\pi i}{n_j}} & e^{-2(n_j-1)\frac{2\pi i}{n_j}} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & e^{-(n_j-2)\frac{2\pi i}{n_j}} & e^{-(n_j-2)2\frac{2\pi i}{n_j}} & \dots & e^{-(n_j-2)^2\frac{2\pi i}{n_j}} & e^{-(n_j-1)(n_j-2)\frac{2\pi i}{n_j}} \\ 1 & e^{-(n_j-1)\frac{2\pi i}{n_j}} & e^{-2(n_j-1)\frac{2\pi i}{n_j}} & \dots & e^{-(n_j-2)(n_j-1)\frac{2\pi i}{n_j}} & e^{-(n_j-1)^2\frac{2\pi i}{n_j}} \end{pmatrix}$$

and the diagonal entries of the corresponding diagonal eigenvalue matrix $\mathbf{\Lambda}_j^B$ are

$$(27) \quad \Lambda_{j,l}^B = \frac{\mathbf{K}_{j,(1,1)}^B + \mathbf{K}_{j,(1,n_j)}^B e^{-(l-1)\frac{2\pi i}{n_j}} + \mathbf{K}_{j,(1,2)}^B e^{-(l-1)(n_j-1)\frac{2\pi i}{n_j}}}{\mathbf{M}_{j,(1,1)}^B + \mathbf{M}_{j,(1,n_j)}^B e^{-(l-1)\frac{2\pi i}{n_j}} + \mathbf{M}_{j,(1,2)}^B e^{-(l-1)(n_j-1)\frac{2\pi i}{n_j}}}$$

for $l = 1, \dots, n_j$. In order to apply the partial solution method the eigenvectors are normalized so that they satisfy the conditions:

$$(28) \quad \mathbf{V}_1^T \mathbf{M}_1 \mathbf{V}_1 = \mathbf{I}_1 \quad \text{and} \quad \mathbf{V}_1^T \mathbf{K}_1 \mathbf{V}_1 = \mathbf{\Lambda}_1^A,$$

$$(29) \quad \mathbf{W}_1^T \mathbf{M}_1^B \mathbf{W}_1 = \mathbf{I}_1 \quad \text{and} \quad \mathbf{W}_1^T \mathbf{K}_1^B \mathbf{W}_1 = \mathbf{\Lambda}_1^B.$$

This can be achieved by multiplying $\mathbf{V}_1$ and $\mathbf{W}_1$ with the vectors $\mathbf{s}_1^A$ and $\mathbf{s}_1^B$ of length n_1 , respectively. We denote the general vectors by $\mathbf{s}_j^A$ and $\mathbf{s}_j^B$ of length n_j . The entries of $\mathbf{s}_j^A$ are given by

$$(30) \quad \mathbf{s}_{j,l}^A = 1/\|\mathbf{M}_j\|_{\mathbf{V}_{j,l}}$$

with the componentwise matrix norms

$$(31) \quad \|\mathbf{M}_j\|_{\mathbf{V}_{j,l}} = \left(\mathbf{V}_{j,l}^T \mathbf{M}_j \mathbf{V}_{j,l} \right)^{1/2},$$

which are weighted by the l -th eigenvectors of $\mathbf{V}_j$ for $l = 1, \dots, n_j$. Similarly, the entries of $\mathbf{s}_j^B$ are given by

$$(32) \quad \mathbf{s}_{j,l}^B = 1/\|\mathbf{M}_j^B\|_{\mathbf{W}_{j,l}}$$

with the componentwise matrix norms

$$(33) \quad \|\mathbf{M}_j^B\|_{\mathbf{W}_{j,l}} = \left(\mathbf{W}_{j,l}^T \mathbf{M}_j^B \mathbf{W}_{j,l} \right)^{1/2},$$

which are weighted by the l -th eigenvectors of $\mathbf{W}_j$ for $l = 1, \dots, n_j$. In (28) and (29), $\mathbf{I}_1$ denotes the identity matrix of size $n_1 \times n_1$. In the following, $\mathbf{I}_j$ and $\mathbf{I}_{jk}$ denote the identity matrices of sizes $n_j \times n_j$ and $(n_j \times n_k) \times (n_j \times n_k)$, respectively. The eigenvector matrices $\mathbf{V}_1$ and $\mathbf{W}_1$ are used for solving the partial solution problems when needed in the steps of the solver. However, the generalized eigenvalue problems (23) and (24) have to be solved only once during the solution process – in the initialization. The conditions (28) and (29) also lead to a convenient representation for the inverses of the system matrices $\mathbf{A}$ and $\mathbf{B}$, which is discussed in Subsections 4.1 and 5.1 for the respective two-dimensional and three-dimensional case. In the next two sections, we discuss the two-dimensional and three-dimensional problems in more detail separately, since the efficient implementation of the initialization process and the steps of the fast solver differs in both cases.

4. THE TWO-DIMENSIONAL CASE

4.1. Preliminaries for the two-dimensional problem.

4.1.1. *Reformulation of problem matrices $\mathbf{A}$ and $\mathbf{B}$.* The matrix $\mathbf{B}$ for the auxiliary problem (14) in the two-dimensional case is given by

$$(34) \quad \mathbf{B} = (\mathbf{K}_1^B - \omega^2 \mathbf{M}_1^B) \otimes \mathbf{M}_2 + \mathbf{M}_1^B \otimes \mathbf{K}_2.$$

Using the conditions (29) as follows

$$(35) \quad \begin{aligned} \mathbf{B} &= (\mathbf{W}_1^{-T} \mathbf{\Lambda}_1^B \mathbf{W}_1^{-1} - \omega^2 \mathbf{W}_1^{-T} \mathbf{I}_1 \mathbf{W}_1^{-1}) \otimes \mathbf{M}_2 + (\mathbf{W}_1^{-T} \mathbf{I}_1 \mathbf{W}_1^{-1}) \otimes \mathbf{K}_2 \\ &= (\mathbf{W}_1^{-T} (\mathbf{\Lambda}_1^B - \omega^2 \mathbf{I}_1) \mathbf{W}_1^{-1}) \otimes \mathbf{M}_2 + (\mathbf{W}_1^{-T} \mathbf{I}_1 \mathbf{W}_1^{-1}) \otimes \mathbf{K}_2 \\ &= (\mathbf{W}_1^{-T} \otimes \mathbf{I}_2) ((\mathbf{\Lambda}_1^B - \omega^2 \mathbf{I}_1) \otimes \mathbf{M}_2 + \mathbf{I}_1 \otimes \mathbf{K}_2) (\mathbf{W}_1^{-1} \otimes \mathbf{I}_2), \end{aligned}$$

the inverse of $\mathbf{B}$ can be represented by

$$(36) \quad \mathbf{B}^{-1} = (\mathbf{W}_1 \otimes \mathbf{I}_2) \mathbf{H}_B^{-1} (\mathbf{W}_1^T \otimes \mathbf{I}_2),$$

where

$$(37) \quad \mathbf{H}_B = (\mathbf{\Lambda}_1^B - \omega^2 \mathbf{I}_1) \otimes \mathbf{M}_2 + \mathbf{I}_1 \otimes \mathbf{K}_2.$$

Similarly using the conditions (28), we obtain for the system matrix $\mathbf{A}$ the following representation:

$$(38) \quad \mathbf{A}^{-1} = (\mathbf{V}_1 \otimes \mathbf{I}_2) \mathbf{H}_A^{-1} (\mathbf{V}_1^T \otimes \mathbf{I}_2),$$

where

$$(39) \quad \mathbf{H}_A = (\mathbf{\Lambda}_1^A - \omega^2 \mathbf{I}_1) \otimes \mathbf{M}_2 + \mathbf{I}_1 \otimes \mathbf{K}_2.$$

4.1.2. *LU decomposition.* Linear systems with the block diagonal matrices $\mathbf{H}_A$ and $\mathbf{H}_B$ are solved with a direct method. The matrices $\mathbf{H}_A$ and $\mathbf{H}_B$ consist of N diagonal blocks each of size N . In the following, let us discuss the method applied on the matrix $\mathbf{H}_B$ (given by (37)), since it applies analogously for $\mathbf{H}_A$. First, the LU decomposition of $\mathbf{H}_B$ is computed as follows

$$(40) \quad \mathbf{H}_B = \mathbf{L}_B \mathbf{U}_B.$$

Then the linear system

$$(41) \quad \mathbf{H}_B \mathbf{y} = \mathbf{L}_B \mathbf{U}_B \mathbf{y} = \mathbf{r}$$

is solved by solving the respective two subproblems

$$(42) \quad \mathbf{L}_B \mathbf{z} = \mathbf{r} \quad \text{and} \quad \mathbf{U}_B \mathbf{y} = \mathbf{z}$$

consecutively in the application of the fast solver. Note that $\mathbf{r}$ denotes now some right-hand side which is in the different steps of the solver also different. However, we will describe that in more detail in the next subsection. We denote by

$$(43) \quad \mathbf{H}_A = \mathbf{L}_A \mathbf{U}_A$$

the LU decomposition corresponding to $\mathbf{H}_A$. The structure of $\mathbf{H}_B$ and $\mathbf{H}_A$ is essential for the fast application of the direct solver. The diagonal blocks of these matrices are tridiagonal which makes the LU decomposition fast for them. The computational complexity is optimal $\mathcal{O}(N)$.

4.2. Fast solver in the two-dimensional case.

4.2.1. *Step 1.* The auxiliary problem

$$(44) \quad \mathbf{B} \mathbf{v} = \mathbf{B} \begin{pmatrix} \mathbf{v}_b \\ \mathbf{v}_r \end{pmatrix} = \mathbf{f}$$

is solved, but only $\mathbf{v}_b$ and not $\mathbf{v}_r$ is computed. For that, the FFT $\hat{\mathbf{f}}$ of the right-hand side $\mathbf{f}$ is computed first, where its coefficients $\hat{f}_k$ are given as follows

$$(45) \quad \hat{f}_k = \sum_{l=1}^N e^{-2\pi i \frac{(l-1)(k-1)}{N}} f_l$$

for all $k = 1, \dots, N$ and normalization might be needed, which means that $\hat{\mathbf{f}}$ is multiplied by the vector $\mathbf{s}_1^B$, where its components are defined in (32) with $j = 1$. Note that computing the FFT

corresponds to the multiplication $\hat{\mathbf{f}} = (\mathbf{W}_1^T \otimes \mathbf{I}_2) \mathbf{f}$. The vector $\hat{\mathbf{f}}$ is saved, since it will be needed in Step 3 as well. Next, we apply the LU decomposition (41) with the right-hand side $\hat{\mathbf{f}}$

$$(46) \quad \mathbf{L}_B \mathbf{z}_1 = \hat{\mathbf{f}} \quad \text{and} \quad \mathbf{U}_B \tilde{\mathbf{z}}_1 = \mathbf{z}_1.$$

Performing the inverse FFT on the resulting vector $\tilde{\mathbf{z}}_1$ would provide both $\mathbf{v}_b$ and $\mathbf{v}_r$, which would correspond to the multiplication $\mathbf{v} = (\mathbf{W}_1 \otimes \mathbf{I}_2) \tilde{\mathbf{z}}_1$. However, since we only need $\mathbf{v}_b$, instead of that, we multiply the vector $\tilde{\mathbf{z}}_1$ by the matrix $\mathbf{W}_1$ by taking advantage of the sparsity of the desired components $\mathbf{v}_b$. More precisely, we multiply $\tilde{\mathbf{z}}_1$ from the left side by the eigenvectors of $\mathbf{W}_1$ which correspond only to the boundary Γ_B denoted by the matrix $\mathbf{W}_1^b$ of size $|b| \times n_1$ leading to

$$(47) \quad \mathbf{v}_b = (\mathbf{W}_1^b \otimes \mathbf{I}_2) \tilde{\mathbf{z}}_1,$$

which resembles the representation (36) for $\mathbf{B}^{-1}$. The computational complexity of Step 1 is $\mathcal{O}(N \log N)$.

4.2.2. *Step 2.* We introduce the additional vector $\mathbf{w}$ defined as $\mathbf{w} = \mathbf{u} - \mathbf{v}$. Since

$$(48) \quad \mathbf{A}\mathbf{w} = \mathbf{A}\mathbf{u} - \mathbf{A}\mathbf{v} = \mathbf{f} - \mathbf{A}\mathbf{v} = \mathbf{B}\mathbf{v} - \mathbf{A}\mathbf{v}$$

and (21), we obtain the following problem:

$$(49) \quad \mathbf{A}\mathbf{w} = \mathbf{A} \begin{pmatrix} \mathbf{w}_b \\ \mathbf{w}_r \end{pmatrix} = (\mathbf{B} - \mathbf{A})\mathbf{v} = \begin{pmatrix} \mathbf{C}_{bb}\mathbf{v}_b \\ 0 \end{pmatrix},$$

where $\mathbf{C}_{bb}$ is defined by (22). We solve the problem (49) by using the representation (38), but compute again only $\mathbf{w}_b$ and not $\mathbf{w}_r$, since the right-hand side as well as the desired components of the solution are both sparse. First, we compute

$$(50) \quad \mathbf{g}_b = (\mathbf{V}_1^{b^T} \otimes \mathbf{I}_2) \mathbf{C}_{bb}\mathbf{v}_b,$$

where the matrix $\mathbf{V}_1^b$ of size $|b| \times n_1$ denotes the eigenvectors of $\mathbf{V}_1$ which correspond only to the boundary Γ_B . Then by using the LU decomposition (43) we solve

$$(51) \quad \mathbf{L}_A \mathbf{z}_2 = \mathbf{g}_b \quad \text{and} \quad \mathbf{U}_A \tilde{\mathbf{z}}_2 = \mathbf{z}_2,$$

and, finally, we solve

$$(52) \quad \mathbf{w}_b = (\mathbf{V}_1^b \otimes \mathbf{I}_2) \tilde{\mathbf{z}}_2,$$

which resembles the representation (38) for $\mathbf{A}^{-1}$ but corresponds only to the boundary Γ_B . The computational complexity of Step 2 is of optimal order $\mathcal{O}(N)$.

4.2.3. *Step 3.* Finally, in order to obtain the solution $\mathbf{u}$ of the original problem (9), we solve now the problem

$$(53) \quad \mathbf{B}\mathbf{u} = \mathbf{f} + (\mathbf{B} - \mathbf{A})(\mathbf{v} + \mathbf{w}) = \mathbf{f} + \begin{pmatrix} \mathbf{C}_{bb}(\mathbf{v}_b + \mathbf{w}_b) \\ 0 \end{pmatrix}$$

due to $\mathbf{B}\mathbf{u} = \mathbf{A}\mathbf{u} + \mathbf{B}\mathbf{u} - \mathbf{A}\mathbf{u}$. Since we have already computed the Fourier transformation $\hat{\mathbf{f}}$ of $\mathbf{f}$ in Step 1 by (45), we only need to compute the Fourier transformation of the second term of the right-hand side of equation (53) and due to sparsity again only the part corresponding to the boundary Γ_B as follows

$$(54) \quad \mathbf{h}_b = (\mathbf{W}_1^{b^T} \otimes \mathbf{I}_2) \mathbf{C}_{bb}(\mathbf{v}_b + \mathbf{w}_b)$$

leading to the Fourier transformation of the entire right-hand side of equation (53) denoted by $\hat{\mathbf{f}} + \hat{\mathbf{h}}$. Next, we apply the LU decomposition (41) with this right-hand side yielding

$$(55) \quad \mathbf{L}_B \mathbf{z}_3 = \hat{\mathbf{f}} + \hat{\mathbf{h}} \quad \text{and} \quad \mathbf{U}_B \tilde{\mathbf{z}}_3 = \mathbf{z}_3.$$

In the last step all resulting components are needed (not only the ones corresponding to Γ_B) to obtain the solution $\mathbf{u}$ by applying the inverse Fourier transformation on $\tilde{\mathbf{z}}_3$, where its coefficients $\hat{u}_k$ are given as follows

$$(56) \quad \hat{u}_k = \frac{1}{N} \sum_{l=1}^N e^{2\pi i \frac{(l-1)(k-1)}{N}} \tilde{z}_{3,l}$$

for all $k = 1, \dots, N$. Again the vector $\tilde{\mathbf{z}}_3$ may need to be multiplied by the vector $\mathbf{s}_1^B$ with entries defined in (32) for n_1 before applying the inverse Fourier transformation on it. The computation of the inverse Fourier transformation corresponds to the multiplication

$$(57) \quad \mathbf{u} = (\mathbf{W}_1 \otimes \mathbf{I}_2) \tilde{\mathbf{z}}_3.$$

The computational complexity of Step 3 is $\mathcal{O}(N \log N)$.

5. THE THREE-DIMENSIONAL CASE

5.1. Preliminaries for the three-dimensional problem.

5.1.1. *Matrices A and B.* The matrix $\mathbf{B}$ in the three-dimensional case is given by

$$(58) \quad \mathbf{B} = (\mathbf{K}_1^B - \omega^2 \mathbf{M}_1^B) \otimes \mathbf{M}_2 \otimes \mathbf{M}_3 + \mathbf{M}_1^B \otimes (\mathbf{K}_2 \otimes \mathbf{M}_3 + \mathbf{M}_2 \otimes \mathbf{K}_3).$$

Using the equations (28) and (29), the inverses of $\mathbf{A}$ and $\mathbf{B}$ can be represented analogously as in the two-dimensional case as follows

$$(59) \quad \mathbf{A}^{-1} = (\mathbf{V}_1 \otimes \mathbf{I}_{23}) \mathbf{H}_A^{-1} (\mathbf{V}_1^T \otimes \mathbf{I}_{23})$$

and

$$(60) \quad \mathbf{B}^{-1} = (\mathbf{W}_1 \otimes \mathbf{I}_{23}) \mathbf{H}_B^{-1} (\mathbf{W}_1^T \otimes \mathbf{I}_{23}),$$

where

$$(61) \quad \mathbf{H}_A = ((\mathbf{\Lambda}_1^A - \omega^2 \mathbf{I}_1) \otimes \mathbf{M}_2 + \mathbf{I}_1 \otimes \mathbf{K}_2) \otimes \mathbf{M}_3 + \mathbf{I}_1 \otimes \mathbf{M}_2 \otimes \mathbf{K}_3$$

and

$$(62) \quad \mathbf{H}_B = ((\mathbf{\Lambda}_1^B - \omega^2 \mathbf{I}_1) \otimes \mathbf{M}_2 + \mathbf{I}_1 \otimes \mathbf{K}_2) \otimes \mathbf{M}_3 + \mathbf{I}_1 \otimes \mathbf{M}_2 \otimes \mathbf{K}_3,$$

respectively.

5.1.2. *Applying the two-dimensional fast solver.* Linear systems with the block diagonal matrices $\mathbf{H}_B$ and $\mathbf{H}_A$ are again solved with a direct method. However, it is performed in a different way than for the two-dimensional case, since the LU decomposition is slow for large block tridiagonal problems. More precisely, the efficient implementation in three dimensions contains the application of the two-dimensional fast solver for n_1 subproblems of size $n_2 \times n_3$ including the computation of the partial solution method in x_2 -direction. For that, one needs to solve the following generalized eigenvalue problems during the initialization process:

$$(63) \quad \mathbf{K}_2 \mathbf{V}_2 = \mathbf{M}_2 \mathbf{V}_2 \mathbf{\Lambda}_2^A \quad \text{and} \quad \mathbf{K}_2^B \mathbf{W}_2 = \mathbf{M}_2^B \mathbf{W}_2 \mathbf{\Lambda}_2^B$$

with the matrices $\mathbf{K}_2^B$ and $\mathbf{M}_2^B$ as defined in (17) and (18) but for the x_2 -direction now. Moreover, the diagonal eigenvalue matrices $\mathbf{\Lambda}_2^A$ and $\mathbf{\Lambda}_2^B$ and the eigenvector matrices $\mathbf{V}_2$ and $\mathbf{W}_2$ are formed analogously as for the x_1 -direction only the problem size changes to n_2 . For that, the generalized eigenvalue problem with circulant matrices for a general problem size n_j was defined in (25) and the corresponding eigenvector matrix is represented in (26). The eigenvectors are normalized to satisfy the conditions

$$(64) \quad \mathbf{V}_2^T \mathbf{M}_2 \mathbf{V}_2 = \mathbf{I}_2 \quad \text{and} \quad \mathbf{V}_2^T \mathbf{K}_2 \mathbf{V}_2 = \mathbf{\Lambda}_2^A,$$

$$(65) \quad \mathbf{W}_2^T \mathbf{M}_2^B \mathbf{W}_2 = \mathbf{I}_2 \quad \text{and} \quad \mathbf{W}_2^T \mathbf{K}_2^B \mathbf{W}_2 = \mathbf{\Lambda}_2^B.$$

This can be achieved by multiplying $\mathbf{V}_2$ and $\mathbf{W}_2$ with the vectors $\mathbf{s}_2^A$ and $\mathbf{s}_2^B$ of length n_2 , respectively, which are introduced in (30)–(33). The fast solver includes the application of the

partial solution method n_1 times for the following two-dimensional $n_2 \times n_3$ subproblems in x_2 -direction corresponding to the matrices (61) and (62):

$$(66) \quad \mathbf{A}_{A,l} = (\mathbf{K}_2 - \underbrace{(\omega^2 - \Lambda_{1,l}^A)}_{=:p_A} \mathbf{M}_2) \otimes \mathbf{M}_3 + \mathbf{M}_2 \otimes \mathbf{K}_3$$

and

$$(67) \quad \mathbf{A}_{B,l} = (\mathbf{K}_2 - \underbrace{(\omega^2 - \Lambda_{1,l}^B)}_{=:p_B} \mathbf{M}_2) \otimes \mathbf{M}_3 + \mathbf{M}_2 \otimes \mathbf{K}_3,$$

where $l = 1, \dots, n_1$. This yields the direct solution for $\mathbf{H}_A^{-1}$ and $\mathbf{H}_B^{-1}$. Note that the subproblems (66) and (67) reflect the structure of (61) and (62), respectively, in the framework of the two-dimensional problem (10). Now the parameters p_A and p_B have been introduced in (66) and (67), respectively. The corresponding auxiliary problems are given by

$$(68) \quad \mathbf{B}_{A,l} = (\mathbf{K}_2^B - p_A \mathbf{M}_2^B) \otimes \mathbf{M}_3 + \mathbf{M}_2^B \otimes \mathbf{K}_3,$$

and

$$(69) \quad \mathbf{B}_{B,l} = (\mathbf{K}_2^B - p_B \mathbf{M}_2^B) \otimes \mathbf{M}_3 + \mathbf{M}_2^B \otimes \mathbf{K}_3,$$

which now reflect the structure of (61) and (62), respectively, in the framework of the two-dimensional problem (34). As defined in (27), $\Lambda_{1,l}^B$ is the l th-diagonal entry of the diagonal eigenvalue matrix Λ_1^B . Analogously, we have denoted by $\Lambda_{1,l}^A$ the l th-diagonal entry of the diagonal eigenvalue matrix Λ_1^A .

In summary, the two-dimensional problems (66) and (67) are solved for all $l = 1, \dots, n_1$ by applying n_1 times the two-dimensional FFT based fast solver from Subsection 4.2 using the auxiliary two-dimensional problems (68) and (69). The computational complexity of this step is now $\mathcal{O}(N \log N)$.

5.2. Fast solver in the three-dimensional case. The three-dimensional solver has the same three main steps, but now instead of applying LU decomposition we apply the two-dimensional solver as described in the previous subsection. We present all the main steps of the solver similarly as for the two-dimensional case in Subsection 4.2 but in less detail. Hence, we refer the reader also to Subsection 4.2 for more details.

5.2.1. *Step 1.* The auxiliary problem (44)

$$\mathbf{B}\mathbf{v} = \mathbf{B} \begin{pmatrix} \mathbf{v}_b \\ \mathbf{v}_r \end{pmatrix} = \mathbf{f}$$

is solved, but only $\mathbf{v}_b$ and not $\mathbf{v}_r$ is computed. For that, we need to compute $\hat{\mathbf{f}} = (\mathbf{W}_1^T \otimes \mathbf{I}_{23}) \mathbf{f}$ first, which is equivalent to the computation of the FFT $\hat{\mathbf{f}}$ for the right-hand side $\mathbf{f}$, where its coefficients $\hat{f}_k$ are given in (45) for all $k = 1, \dots, N$. The vector $\hat{\mathbf{f}}$ can be normalized by multiplying it with the vector $\mathbf{s}_1^B$ of length n_1 with components defined in (32). The vector $\hat{\mathbf{f}}$ is saved, since it will be needed in Step 3 as well. Next, we apply n_1 times the two-dimensional solver for the $n_2 \times n_3$ problems (67) with the auxiliary problems (69) leading to the solution of the problem

$$(70) \quad \mathbf{H}_B \tilde{\mathbf{z}}_1 = \hat{\mathbf{f}}.$$

Since we only need $\mathbf{v}_b$, we now multiply the vector $\tilde{\mathbf{z}}_1$ by the matrix $\mathbf{W}_1$ by taking advantage of the sparsity of the desired components $\mathbf{v}_b$. More precisely, we multiply $\tilde{\mathbf{z}}_1$ from the left side by the components of the eigenvectors of $\mathbf{W}_1$ which correspond only to the boundary Γ_B denoted by $\mathbf{W}_1^b$ leading to

$$(71) \quad \mathbf{v}_b = (\mathbf{W}_1^b \otimes \mathbf{I}_{23}) \tilde{\mathbf{z}}_1,$$

which resembles the representation (60) for $\mathbf{B}^{-1}$.

5.2.2. *Step 2.* We introduce the additional vector $\mathbf{w}$ defined as $\mathbf{w} = \mathbf{u} - \mathbf{v}$. Since (48) and (21), we derive at problem (49)

$$\mathbf{A}\mathbf{w} = \mathbf{A} \begin{pmatrix} \mathbf{w}_b \\ \mathbf{w}_r \end{pmatrix} = (\mathbf{B} - \mathbf{A})\mathbf{v} = \begin{pmatrix} \mathbf{C}_{bb}\mathbf{v}_b \\ 0 \end{pmatrix}.$$

We solve this problem by using the representation (59), but compute again only $\mathbf{w}_b$ and not $\mathbf{w}_r$, since the right-hand side as well as the desired components of the solution are both sparse. First, we solve

$$(72) \quad \mathbf{g}_b = (\mathbf{V}_1^{bT} \otimes \mathbf{I}_{23}) \mathbf{C}_{bb}\mathbf{v}_b,$$

then by applying n_1 times the two-dimensional solver for the $n_2 \times n_3$ subproblems (66) with the auxiliary problems (68) leading to the solution of the problem

$$(73) \quad \mathbf{H}_A \tilde{\mathbf{z}}_2 = \mathbf{g}_b.$$

Finally, we compute

$$(74) \quad \mathbf{w}_b = (\mathbf{V}_1^b \otimes \mathbf{I}_{23}) \tilde{\mathbf{z}}_2,$$

which resembles the representation (59) for $\mathbf{A}^{-1}$ but corresponds only to the boundary Γ_B .

5.2.3. *Step 3.* The last step yields the solution $\mathbf{u}$ of the original problem (9). First, we solve the problem (53)

$$\mathbf{B}\mathbf{u} = \mathbf{f} + (\mathbf{B} - \mathbf{A})(\mathbf{v} + \mathbf{w}) = \mathbf{f} + \begin{pmatrix} \mathbf{C}_{bb}(\mathbf{v}_b + \mathbf{w}_b) \\ 0 \end{pmatrix}$$

due to $\mathbf{B}\mathbf{u} = \mathbf{A}\mathbf{u} + \mathbf{B}\mathbf{u} - \mathbf{A}\mathbf{u}$. Since we have already computed the Fourier transformation $\hat{\mathbf{f}}$ of $\mathbf{f}$ in Step 1 by (45), we only need to compute the Fourier transformation of the second term of the right-hand side of equation (53) and due to sparsity again only the part corresponding to the boundary Γ_B as follows

$$(75) \quad \mathbf{h}_b = (\mathbf{W}_1^{bT} \otimes \mathbf{I}_{23}) \mathbf{C}_{bb}(\mathbf{v}_b + \mathbf{w}_b)$$

leading to the Fourier transformation of the entire right-hand side of equation (53) denoted by $\hat{\mathbf{f}} + \hat{\mathbf{h}}$. Next, we apply n_1 times the two-dimensional solver for the $n_2 \times n_3$ problems (67) with the auxiliary problems (69) leading to the solution of the problem

$$(76) \quad \mathbf{H}_B \tilde{\mathbf{z}}_3 = \hat{\mathbf{f}} + \hat{\mathbf{h}}.$$

In the last step all resulting components are needed (not only the ones corresponding to Γ_B) to obtain the solution $\mathbf{u}$ by applying the inverse Fourier transformation on $\tilde{\mathbf{z}}_3$, where its coefficients $\hat{u}_k$ are defined as in (56) for all $k = 1, \dots, N$. The vector $\tilde{\mathbf{z}}_3$ may be normalized by multiplying it with the vector $\mathbf{s}_1^B$ of length n_1 with the components defined in (32) before applying the inverse Fourier transformation which corresponds to the multiplication

$$(77) \quad \mathbf{u} = (\mathbf{W}_1 \otimes \mathbf{I}_{23}) \tilde{\mathbf{z}}_3.$$

finally leading to the solution $\mathbf{u}$.

In the next section, we present numerical experiments for both the two-dimensional and three-dimensional case.

6. NUMERICAL RESULTS

For $d = 2$ and $d = 3$, the computational domain is chose as $\Omega = [0, 1]^d$ the unit square and unit cube, respectively. The wave number is set $\omega = 2\pi$ for all numerical experiments. The discretization meshes are uniform with respect to each x_j -direction, where the corresponding step sizes are denoted by $h_j = 1/(n_j - 1), j = 1, \dots, d$. The right-hand side is chosen as 0.01 for the first n_1 entries and 1 for all the other entries. In this set of experiments, the efficiency of the fast direct solver is discussed. The numerical experiments have been computed using MATLAB 9.3, R2017b.

In the following, we compare the CPU times in seconds for computing the solution by applying Matlab's backslash and the fast solver presented in this work. In our case, Matlab's backslash uses the sparse direct solver UMFPACK for computing the solution of the sparse linear systems (10) and (11), see [4]. Besides the computational times of applying Matlab's backslash and the FFT based direct solver also the computational times needed in the initialization process are presented. However, the initialization processes differ between the two- and three-dimensional problems, which has already been addressed in Sections 4 and 5. Summing up, in the two-dimensional case the LU decompositions are computed for the matrices $\mathbf{H}_B$ and $\mathbf{H}_A$ defined in (37) and (39), respectively, (see Subsection 4.1.2), whereas the three-dimensional problem is not solved using LU decomposition in three dimensions. In this case, the action of the inverses of $\mathbf{H}_B$ and $\mathbf{H}_A$ defined in (62) and (61), respectively, are computed by applying the fast solver in two dimensions n_1 times (see Subsection 5.1.2). This approach leads to a more efficient implementation in three dimensions.

The largest numerical experiments were computed in three dimensions with 135 005 697 ($= 513^3$) unknowns. Note that this size is already too large regarding the computational memory in Matlab for setting up the whole matrix $\mathbf{A}$ as defined in (11), which would be needed in order to apply Matlab's backslash. However, the fast solver presented in this work still computes the solution in a reasonable amount of time without using too much computational memory, since the fast solver does not need to form the whole matrix $\mathbf{A}$, but only of the left upper block (22) denoted by $\mathbf{C}_{bb}$ of the matrix $\mathbf{B} - \mathbf{A}$ defined in (21).

Remark 3. *All numerical experiments presented in this section were performed on a laptop with Intel(R) Core(TM) i5-6267U CPU @ 2.90GHz processor and 16 GB 2133 MHz LPDDR3 memory.*

6.1. Numerical results for the two-dimensional case. For the first set of numerical experiments, we choose $n = n_1 = n_2$. The CPU times in seconds for the initialization process as well as the application of the fast solver and Matlab's backslash for comparison are presented in Table 1 for different values of n . The largest numerical experiments in two dimensions have been computed for $n = 2049$ with 4 198 401 unknowns. As can be observed in Table 1, the CPU times applying Matlab's backslash are about the same size as for the initialization, whereas the CPU times of the FFT based fast solver grow with $\mathcal{O}(N \log N)$.

n	65	129	257	513	1025	2049
Initialization	0.07	0.09	0.31	1.74	13.38	118.99
Matlab's backslash	0.04	0.14	0.50	2.64	12.42	93.32
Fast solver	0.01	0.02	0.06	0.23	1.05	4.58

TABLE 1. CPU times in seconds for different values of $n = n_1 = n_2$ in the two-dimensional case

Table 2 presents the CPU times in seconds for various combinations of n_1 and n_2 in the two-dimensional case. More precisely, we compare here the computational times with respect to a large difference in the magnitude of n_1 and n_2 . Both cases are discussed $n_1 \gg n_2$ and $n_2 \gg n_1$. As one can observe in Table 2 it does not influence the computational times of the solver whether $n_1 \gg n_2$ or $n_2 \gg n_1$. For $n_1 = 65$ and $n_2 = 2049$, the fast solver's CPU time was 0.10 seconds, and for $n_1 = 2049$ and $n_2 = 65$, it was 0.12 seconds. However, if n_1 is very large, it influences the computational times of the initialization process, more precisely, of solving the generalized eigenvalue problem (23) performed by applying Matlab's function `eig`, which requires the full representations of the matrices $\mathbf{K}_1$ and $\mathbf{M}_1$, which are otherwise stored as sparse matrices. These increased computational times can be observed in the last two columns of Table 2. We note here that the solution method described in [11] would be faster for solving this generalized eigenvalue problem.

6.2. Numerical results for the three-dimensional case. In the three-dimensional case, we chose $n_j = n$ for all $j = 1, 2, 3$ for the first set of numerical experiments again. The CPU times

n_1	65	65	2049	2049
n_2	65	2049	65	2049
Initialization	0.07	0.17	110.83	118.99
Matlab's backslash	0.04	0.98	1.07	93.32
Fast solver	0.01	0.10	0.12	4.58

TABLE 2. CPU times in seconds for various combinations of n_1 and n_2 in the two-dimensional case

in seconds for the initialization process as well as the application of the fast solver and Matlab's backslash for comparison are presented in Table 3 for different values of n . As can be observed from Table 3, the computational times for applying Matlab's backslash rise very quickly. For $n = 65$, its CPU time was 608.12 seconds, whereas the fast solver's CPU time was only 2.02 seconds. Then already for $n = 129$ with 2 146 689 unknowns, Matlab runs out of memory when the matrix $\mathbf{A}$ is formed which makes it impossible to solve the problem (9) by applying Matlab's backslash. However, since the FFT based fast solver only needs the setting up of the left upper block $\mathbf{C}_{bb}$ of the matrix $\mathbf{B} - \mathbf{A}$ and not of the entire matrix $\mathbf{A}$, the solver can be applied solving the problem (9) up to size $n = 513$ with 135 005 697 unknowns (Remark 3). Moreover, we can observe from Table 3 that the CPU times applying Matlab's backslash are already much larger for $n = 33$ than for the initialization process and the application of the fast solver. Table 3 shows that the CPU times of applying the FFT based fast solver are nearly optimal order of complexity $\mathcal{O}(N \log N)$.

n	9	17	33	65	129	257	513
Initialization	0.07	0.07	0.08	0.45	0.41	2.16	17.33
Matlab's backslash	0.02	0.28	6.39	572.91	—	—	—
Fast solver	0.09	0.12	0.21	1.03	8.43	69.55	672.27

TABLE 3. CPU times in seconds for different values of $n = n_1 = n_2 = n_3$ in the three-dimensional case

Table 4 presents the CPU times in seconds for various combinations of n_j , $j = 1, 2, 3$, in the three-dimensional case similar to Table 2 for two dimensions comparing computational times with respect to a large difference in the magnitudes of n_j . Again we discuss all possible combinations, e.g., $n_1 \gg n_2, n_3$ or $n_1, n_2 \gg n_3$. The third and fourth column of Table 4 comparing $n_2 \gg n_1, n_3$ and $n_3 \gg n_1, n_2$ show that the CPU times in applying Matlab's backslash are similar (1.06 and 0.96 in the third and fourth column, respectively) and also applying the fast solver (0.22 and 0.25 in the third and fourth column, respectively). However, note that the FFT based direct solver is already four times faster than Matlab's backslash for this set of values. Also, the sixth, seventh and eighth column present similar numerical results, but Matlab's backslash cannot be applied anymore for these combinations, since Matlab runs out of memory when forming the matrix $\mathbf{A}$. Note that the computational times for the initialization process needed to apply the FFT based direct solver take only around 2 seconds for these sets of values.

Remark 4. For a large n , applying Matlab's LU decomposition `lu` on the subproblems (66)–(69) as well as Matlab's `kron` function are the most time consuming functions in the computational process. Based on this it can be expected that, for example, a C++ implementation would be essentially faster than the Matlab implementation.

7. CONCLUSIONS

In this work, we have derived an FFT based direct solver for solving efficiently the Helmholtz equation in a rectangular domain with an absorbing boundary condition. The model problem and

n_1	9	9	9	513	9	513	513	513
n_2	9	513	9	9	513	9	513	513
n_3	9	9	513	9	513	513	9	513
Initialization	0.07	0.36	0.12	1.91	2.17	2.20	2.40	17.33
Matlab's backslash	0.02	1.06	0.96	0.90	–	–	–	–
Fast solver	0.09	0.22	0.25	0.62	10.95	8.00	7.01	672.27

TABLE 4. CPU times in seconds for various combinations of n_1 and n_2 and n_3 in the three-dimensional case

fast solver are discussed for two- and three-dimensional domains as well as numerical results for both cases are presented. The experiments show nearly optimal order of complexity $\mathcal{O}(N \log N)$ matching the efficiency of the FFT method. In particular, the numerical results in this work show the efficiency of the fast solver compared to Matlab's backslash and also with respect to the computational memory needed to solve the discretized Helmholtz problem for a large number of unknowns.

ACKNOWLEDGMENTS

The authors gratefully acknowledge the financial support by the Academy of Finland under the grant 295897. The authors like to thank Dr. Kazufumi Ito for many fruitful discussions on these fast direct solvers.

REFERENCES

- [1] A. BAMBERGER, P. JOLY, AND J. E. ROBERTS, *Second-order absorbing boundary conditions for the wave equation: a solution for the corner problem*, SIAM J. Numer. Anal., 27 (1990), pp. 323–352.
- [2] A. BANEGAS, *Fast Poisson solvers for problems with sparsity*, Math. Comp., 32 (1978), pp. 441–446.
- [3] B. BUZBEE, F. W. DORR, J. A. GEORGE, AND G. H. GOLUB, *The direct solution of the discrete Poisson equation on irregular regions*, SIAM J. Numer. Anal., 8 (1971), pp. 722–736.
- [4] T. A. DAVIS, *Direct methods for sparse linear systems*, vol. 2, Fundamentals of Algorithms, SIAM, Philadelphia, PA, USA, 2006.
- [5] O. ERNST AND G. H. GOLUB, *A domain decomposition approach to solving the Helmholtz equation with a radiation boundary condition*, Contemp. Math., 157 (1994), pp. 177–192.
- [6] O. G. ERNST, *A finite-element capacitance matrix method for exterior Helmholtz problems*, Numer. Math., 75 (1996), pp. 175–204.
- [7] M. N. GUDDATI AND B. YUE, *Modified integration rules for reducing dispersion in finite element methods*, Comput. Methods Appl. Mech. Engrg., 193 (2004), pp. 275–287.
- [8] E. HEIKKOLA, K. ITO, AND J. TOIVANEN, *A parallel domain decomposition method for the Helmholtz equation in layered media*, submitted for publication, (2018).
- [9] E. HEIKKOLA, Y. A. KUZNETSOV, AND K. N. LIPNIKOV, *Fictitious domain methods for the numerical solution of three-dimensional acoustic scattering problems*, J. Comput. Acoust., 7 (1999), pp. 161–183.
- [10] E. HEIKKOLA, Y. A. KUZNETSOV, P. NEITTAANMÄKI, AND J. TOIVANEN, *Fictitious domain methods for the numerical solution of two-dimensional scattering problems*, J. Comput. Phys., 145 (1998), pp. 89–109.
- [11] E. HEIKKOLA, T. ROSSI, AND J. TOIVANEN, *Fast direct solution of the Helmholtz equation with a perfectly matched layer or an absorbing boundary condition*, Internat. J. Numer. Methods Engrg., 57 (2003), pp. 2007–2025.
- [12] E. HEIKKOLA, T. ROSSI, AND J. TOIVANEN, *A parallel fictitious domain method for the three-dimensional Helmholtz equation*, SIAM J. Sci. Comput., 24 (2003), pp. 1567–1588.
- [13] K. ITO, Z. QIAO, AND J. TOIVANEN, *A domain decomposition solver for acoustic scattering by elastic objects in layered media*, J. Comput. Phys., 227 (2008), pp. 8685–8698.
- [14] Y. A. KUZNETSOV AND A. M. MATSOKIN, *On partial solution of systems of linear algebraic equations*, Sov. J. Numer. Anal. Math. Modelling, 4 (1989), pp. 453–467.
- [15] R. E. LYNCH, J. R. RICE, AND D. H. THOMAS, *Direct solution of partial difference equations by tensor product methods*, Numerische Mathematik, 6 (1964), pp. 185–199.
- [16] T. ROSSI AND J. TOIVANEN, *A nonstandard cyclic reduction method, its variants and stability*, SIAM J. Matrix Anal. Appl., 20 (1999), pp. 628–645.
- [17] T. ROSSI AND J. TOIVANEN, *A parallel fast direct solver for block tridiagonal systems with separable matrices of arbitrary dimension*, SIAM J. Sci. Comput., 20 (1999), pp. 1778–1793.

- [18] P. N. SWARZTRAUBER, *The methods of cyclic reduction, Fourier analysis and the FACR algorithm for the discrete solution of Poisson's equation on a rectangle*, SIAM Rev., 19 (1977), pp. 490–501.
- [19] R. TEZAUER, A. MACEDO, AND C. FARHAT, *Iterative solution of large-scale acoustic scattering problems with multiple right hand-sides by a domain decomposition method with Lagrange multipliers*, Internat. J. Numer. Methods Engrg., 51 (2001), pp. 1175–1193.
- [20] E. TURKEL, *Numerical difficulties solving time harmonic systems*, Nato Sci. S SS III Comput. Systems Sci., 177 (2001), pp. 319–337.
- [21] P. VASSILEVSKI, *Fast algorithm for solving a linear algebraic problem with separable variables*, Dokl. Bolg. Akad. Nauk., 37 (1984), pp. 305–308.

(J. Toivanen) FACULTY OF INFORMATION TECHNOLOGY; UNIVERSITY OF JYVÄSKYLÄ; P.O. Box 35 (AGORA); FIN-40014 JYVÄSKYLÄ; FINLAND

E-mail address: `jari.a.toivanen@jyu.fi`

(M. Wolfmayr) FACULTY OF INFORMATION TECHNOLOGY; UNIVERSITY OF JYVÄSKYLÄ; P.O. Box 35 (AGORA); FIN-40014 JYVÄSKYLÄ; FINLAND

E-mail address: `monika.k.wolfmayr@jyu.fi`